

Cloud Based Data Engineering

Lecture 6

Solve Problems With Hadoop

TF-IDF

Term Frequency – Inverse Document Frequency

TF-IDF (term frequency-inverse document frequency) is a statistical measure that evaluates how relevant a word is to a document in a collection of documents.

Done by multiplying two metrics:

1. how many times a word appears in a document
2. The inverse document frequency of the word across a set of documents.

Applications of TF-IDF

Information retrieval

TF-IDF was invented for document search and can be used to deliver results that are most relevant to what you're searching for.

Keyword Extraction

- Useful for extracting keywords from text
- The highest scoring words of a document are the most relevant to that document

Terminology

1. t – term / word
2. d – document (set of words)
3. N – count of corpus
4. Corpus – the total document set

TF-IDF = Term Frequency (TF) * Inverse Document Frequency (IDF)

Terminology

TF-IDF = Term Frequency (TF) * Inverse Document Frequency (IDF)

$$TF(\textcolor{red}{t}, \textcolor{blue}{d}) = \frac{\textit{count of } \textcolor{red}{t} \textit{ in } \textcolor{blue}{d}}{\textit{number of words in } \textcolor{blue}{d}}$$

$DF(\textcolor{red}{t}) = \textit{occurrence of } \textcolor{red}{t} \textit{ in } N \textit{ documents}$

$$IDF(\textcolor{red}{t}) = \log\left(\frac{N}{DF(\textcolor{red}{t})}\right)$$

$$TF - IDF(\textcolor{red}{t}, \textcolor{blue}{d}) = TF(\textcolor{red}{t}, \textcolor{blue}{d}) \cdot \log\left(\frac{N}{DF(\textcolor{red}{t})}\right)$$

Example

Document 1 It is going to rain today.

Document 2 Today I am not going outside.

Document 3 I am going to watch the season premiere.

Step 1: Counts

Word	Count
going	3
to	2
today	2
i	2
am	2
it	1
is	1
rain	1

Example

Step 2: Find TF

$$TF(t, d) = \frac{\text{count of } t \text{ in } d}{\text{number of words in } d}$$

Word	Document 1
going	$\frac{1}{6} = 0.16$
to	0.16
today	0.16
i	0
am	0
it	0.16
is	0.16
rain	0.16

Example

Step 2: Find TF

$$TF(t, d) = \frac{\text{count of } t \text{ in } d}{\text{number of words in } d}$$

Word	Document 1	Document 2
going	$\frac{1}{6} = 0.16$	0.16
to	0.16	0
today	0.16	0.16
i	0	0.16
am	0	0.16
it	0.16	0
is	0.16	0
rain	0.16	0

Example

Step 2: Find TF

$$TF(\textcolor{red}{t}, \textcolor{blue}{d}) = \frac{\text{count of } \textcolor{red}{t} \text{ in } \textcolor{blue}{d}}{\text{number of words in } \textcolor{blue}{d}}$$

Word	Document 1	Document 2	Document 3
going	$\frac{1}{6} = 0.16$	0.16	0.12
to	0.16	0	0.12
today	0.16	0.16	0
i	0	0.16	0.12
am	0	0.16	0.12
it	0.16	0	0
is	0.16	0	0
rain	0.16	0	0

Example

Step 3: Find IDF

$DF(t)$ = occurrence of t in N documents

$$IDF(t) = \log\left(\frac{N}{DF(t)}\right)$$

Word	Count	IDF
going	3	$\log\left(\frac{3}{3}\right) = 0$
to	2	$\log(1.5) = 0.58$
today	2	$\log(1.5) = 0.58$
i	2	$\log(1.5) = 0.58$
am	2	$\log(1.5) = 0.58$
it	1	$\log(3) = 1.58$
is	1	$\log(3) = 1.58$
rain	1	$\log(3) = 1.58$

Example

Step 4: Find TF-IDF

Word	IDF	Document 1	Document 2	Document 3
going	0	0.16	0.16	0.12
to	0.58	0.16	0	0.12
today	0.58	0.16	0.16	0
i	0.58	0	0.16	0.12
am	0.58	0	0.16	0.12
it	1.58	0.16	0	0
is	1.58	0.16	0	0
rain	1.58	0.16	0	0

Example

Step 4: Find TF-IDF

	going	to	today	i	am	it	is	rain
Doc 1	0	$0.58 \cdot 0.16 = 0.09$	0.09	0	0	0.25	0.25	0.25
Doc 2	0	0	0.09	0.09	0.09	0	0	0
Doc 3	0	$0.58 \cdot 0.12 = 0.06$	0	0.06	0.06	0	0	0

- “it”, “is”, “rain” important for doc1 but not for docs 2 and 3.
- Docs 1 and 2 talks about today

TF-IDF Distributed Approach

Task 1: Word Frequency in Doc

Mapper

Input: (docname, contents)

Output: ((word, docname), 1)

Reducer

Sums counts for word in document

Outputs ((word, docname), n)

TF-IDF Distributed Approach

Task 2: Word Counts For Docs

Mapper

Input: $((\text{word}, \text{docname}), n)$

Output: $(\text{docname}, (\text{word}, n))$

Reducer

Sums frequency of individual n 's in same doc

Feeds original data through

Outputs $((\text{word}, \text{docname}), (n, N))$

TF-IDF Distributed Approach

Task 3: Word Frequency In Corpus

Mapper

Input: $((\text{word}, \text{docname}), (n, N))$

Output: $(\text{word}, (\text{docname}, n, N, 1))$

Reducer

Sums counts for word in corpus

Outputs $((\text{word}, \text{docname}), (n, N, m))$

TF-IDF Distributed Approach

Task 4: Calculate TF-IDF

Mapper

Input: ((word, docname), (n, N, m))

Output ((word, docname), TF*IDF)

Reducer

Just the identity function

Solve Problems With Hadoop

Shortest Path – Graph Theory

Graph Theory

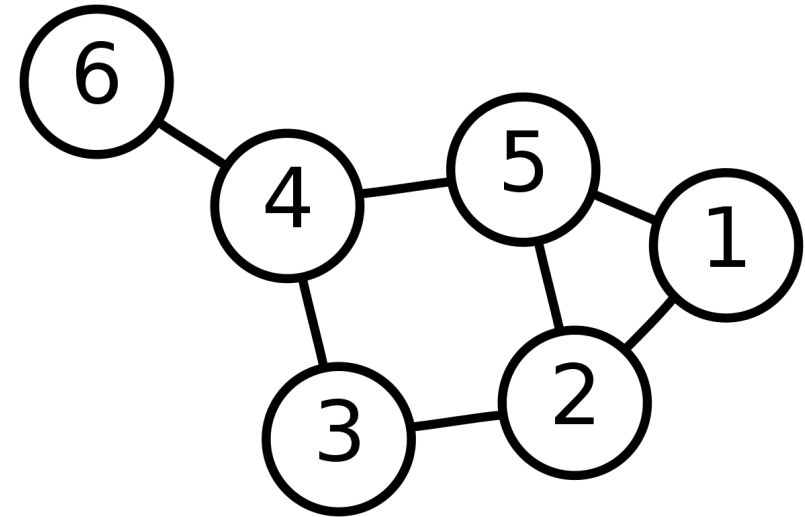
A graph $G = (V, E)$ consists of two finite sets:

1. V – the set of vertices
2. E – the set of edges

For example, in the figure below:

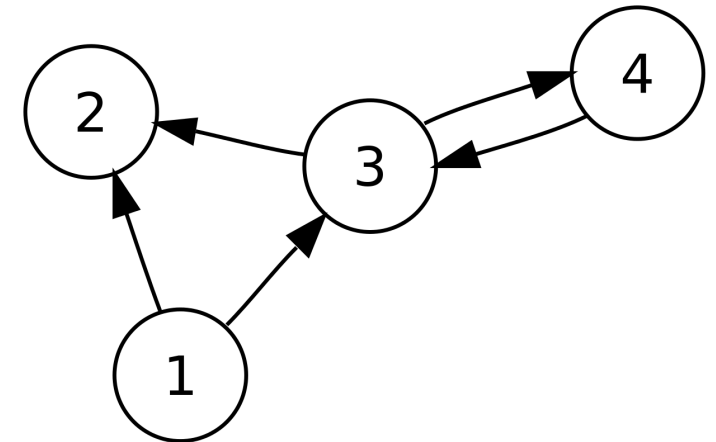
$$V = \{1, 2, 3, 4, 5, 6\}$$

$$E = \{(1, 2), (2, 3), (1, 5) \dots\}$$



Definitions

- **Directed Graph:** each edge has a direction
- **Undirected Graph:** each edge is bidirectional
- **Vertex Degree:** number of edges connected to the vertex, marked as $d(v)$
 - In directed graph, each vertex has d_{out} and d_{in}
- **Path:** set of vertices
- **Weighted Graph:** each edge has a cost.
 - For example, Waze for traffic detector
- **Circle:** a closed path (start and end in the same vertex)



Graphs: Examples

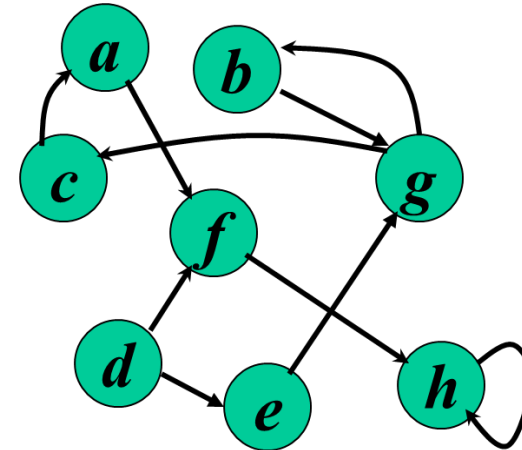
The internet: webpages are represented as nodes and the hyperlinks between those sites are represented as edges.

Social Networks: each person is represented as a vertex, and the edges could represent the friendships between the individuals.

Represent A Graph

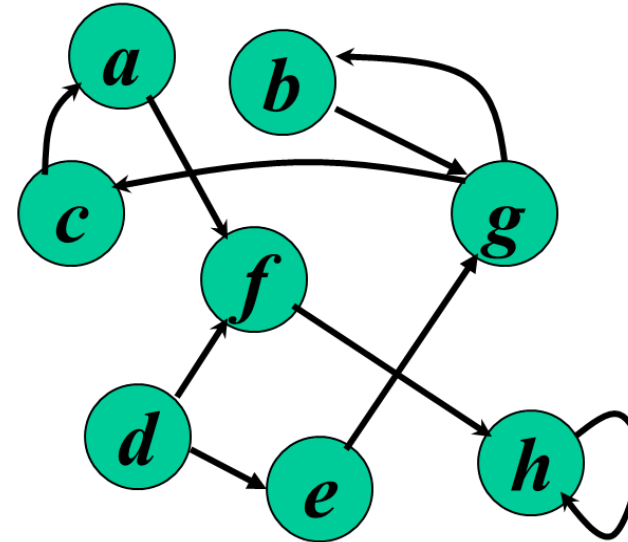
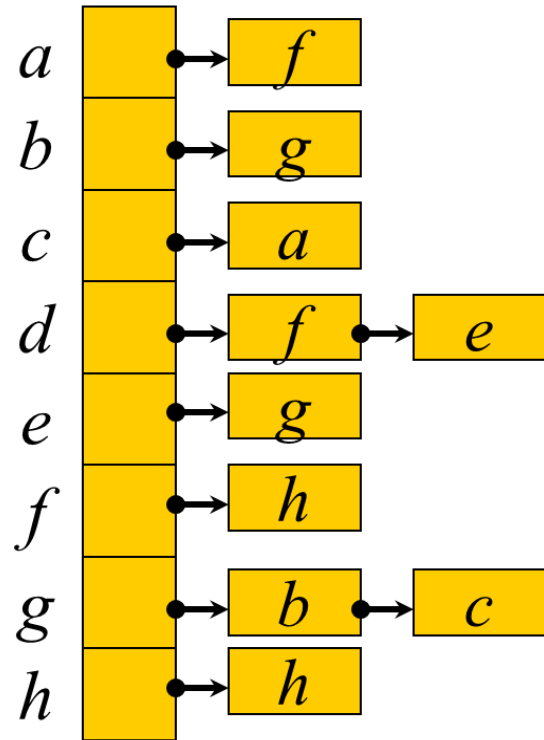
- **Adjacency Matrix:** size $O(|V|^2)$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>
<i>a</i>	0	0	0	0	0	1	0	0
<i>b</i>	0	0	0	0	0	0	1	0
<i>c</i>	1	0	0	0	0	0	0	0
<i>d</i>	0	0	0	0	1	1	0	0
<i>e</i>	0	0	0	0	0	0	1	0
<i>f</i>	0	0	0	0	0	0	0	1
<i>g</i>	0	1	1	0	0	0	0	0
<i>h</i>	0	0	0	0	0	0	0	1



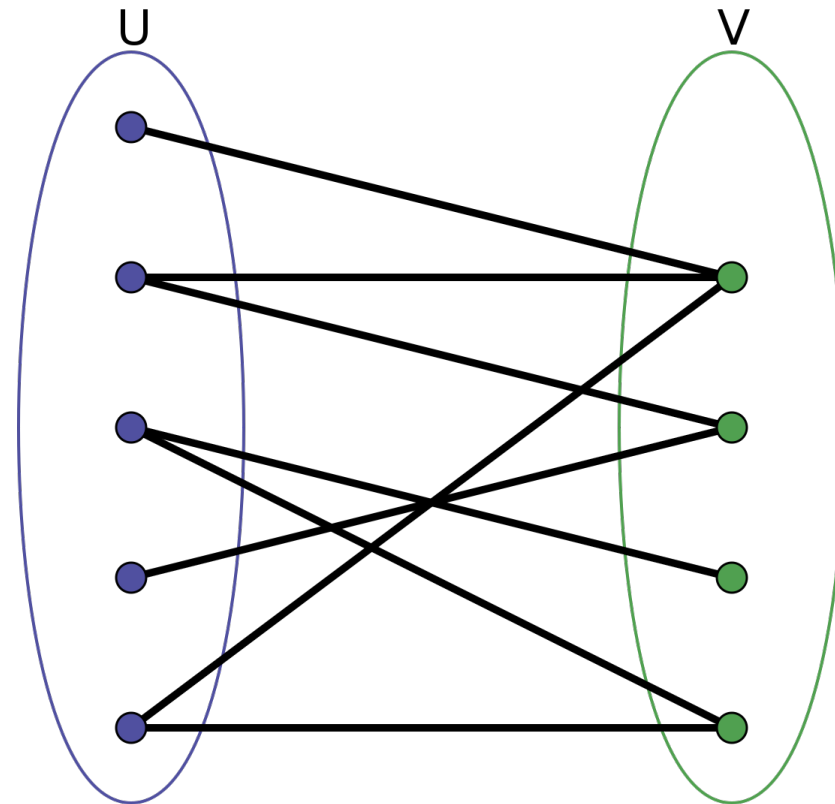
Represent A Graph

- Adjacency Lists (Hash Table + Linked List): $O(|V| + |E|)$



Graph Problems

- Finding shortest paths
 - Routing Internet traffic and UPS trucks
- Finding minimum spanning trees
 - Google Fiber
- Finding Max Flow
 - Scheduling
- Bipartite matching
 - Dating websites



Breadth First Search - BFS

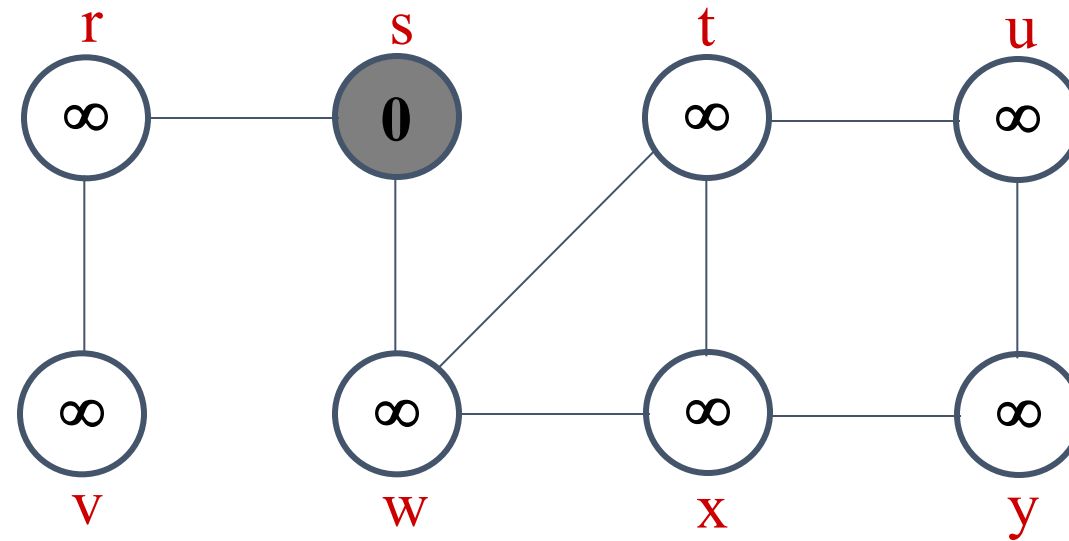
- **Input:** $G = (V, E)$ and a source vertex (marked as s)
- **Output:** the distance of all vertices from s

The BFS algorithm use three colors to track over the vertices:

1. **White** – undiscovered vertices
2. **Gray** – discovered, “currently handled”
3. **Black** – done

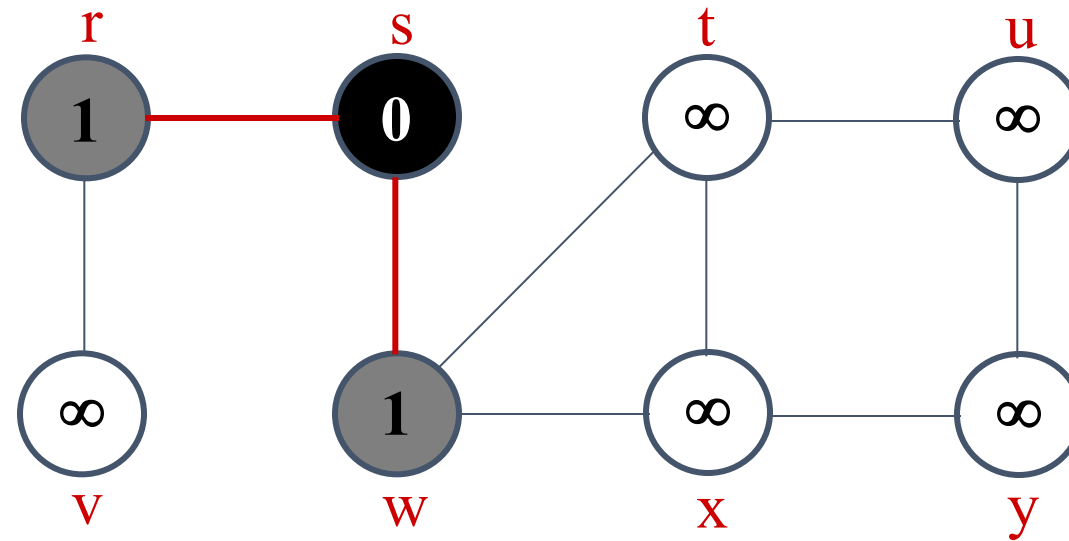
Also, the algorithm use a Queue of discovered vertices.

BFS Example



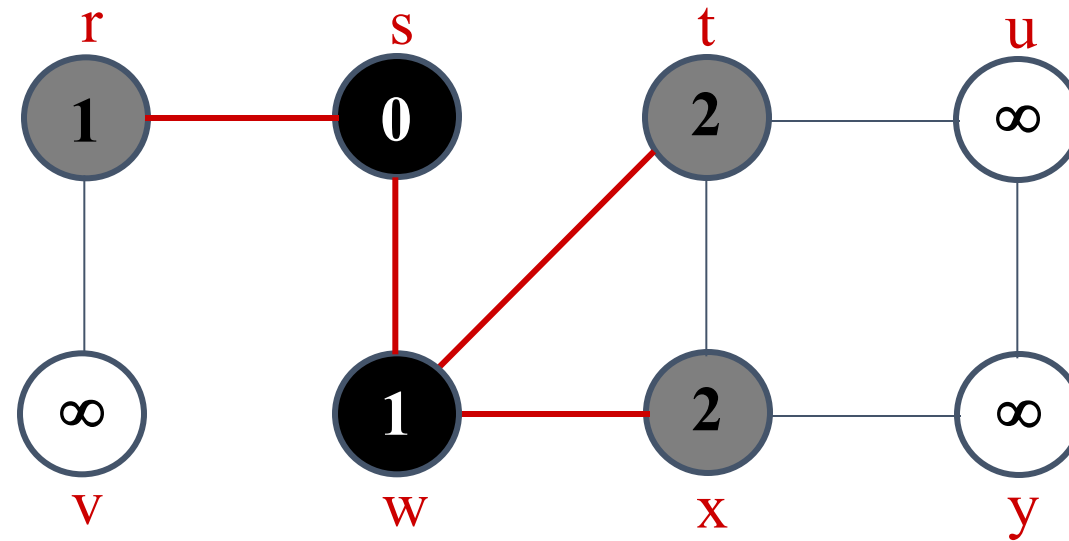
Q: s
0

BFS Example



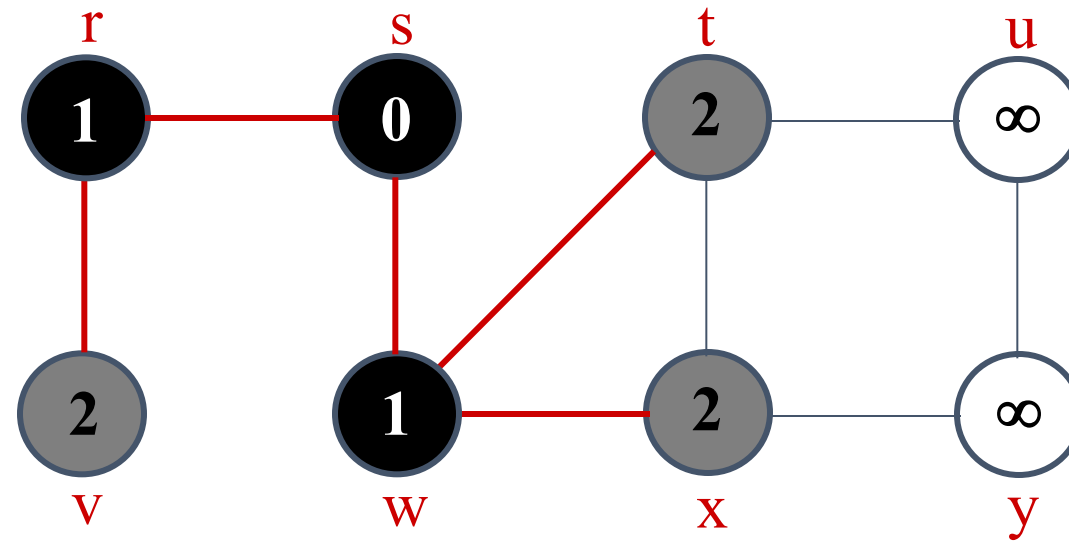
Q: w r
1 1

BFS Example



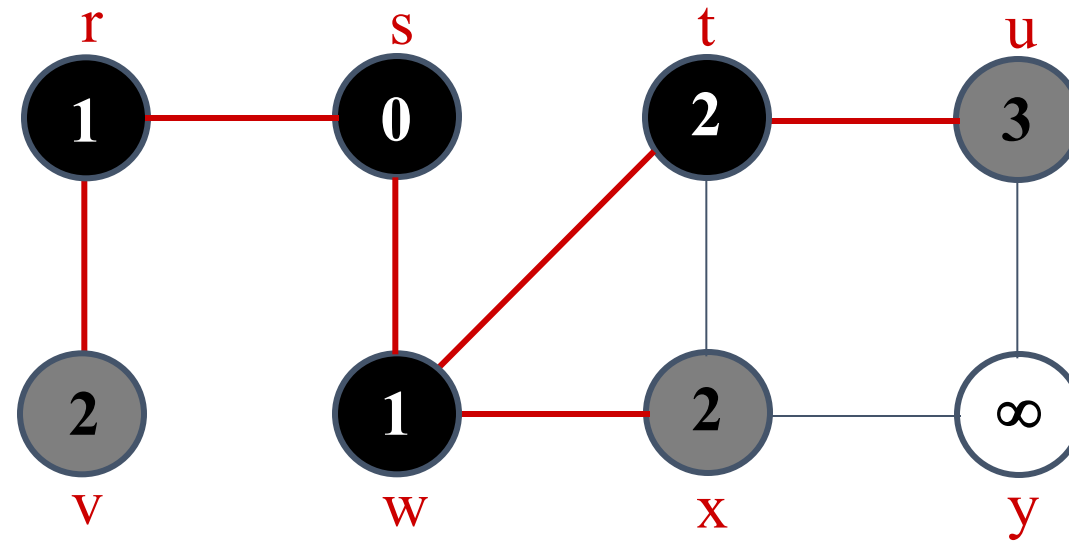
Q:	r	t	x
	1	2	2

BFS Example



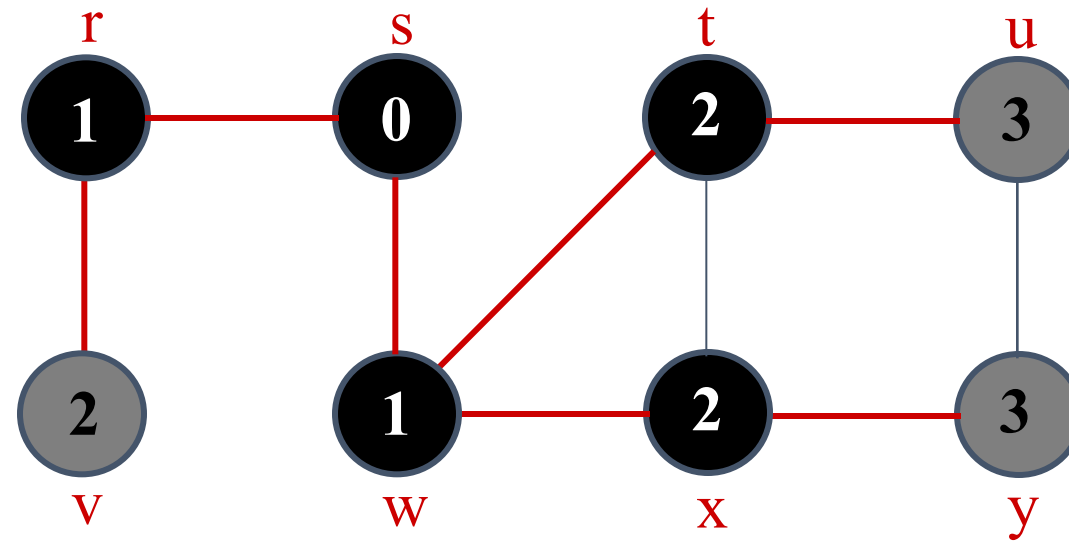
Q: t x v
2 2 2

BFS Example



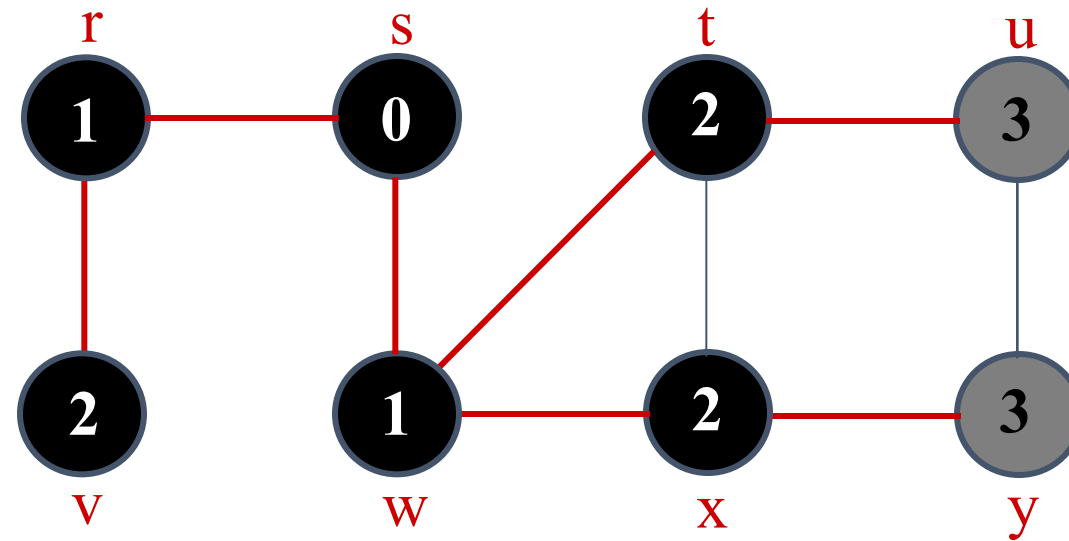
Q: x v u
2 2 3

BFS Example



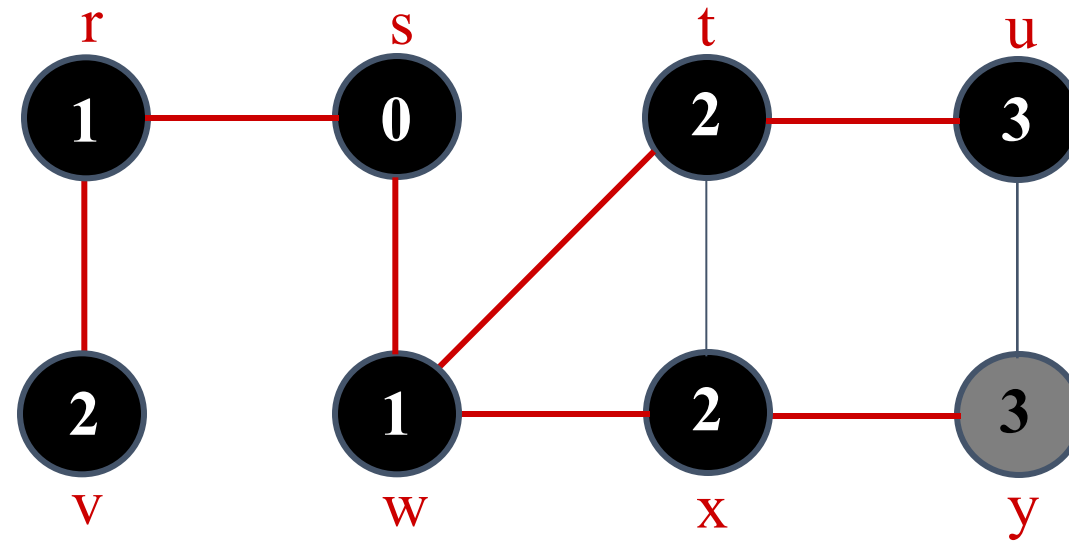
Q: v u y
2 3 3

BFS Example



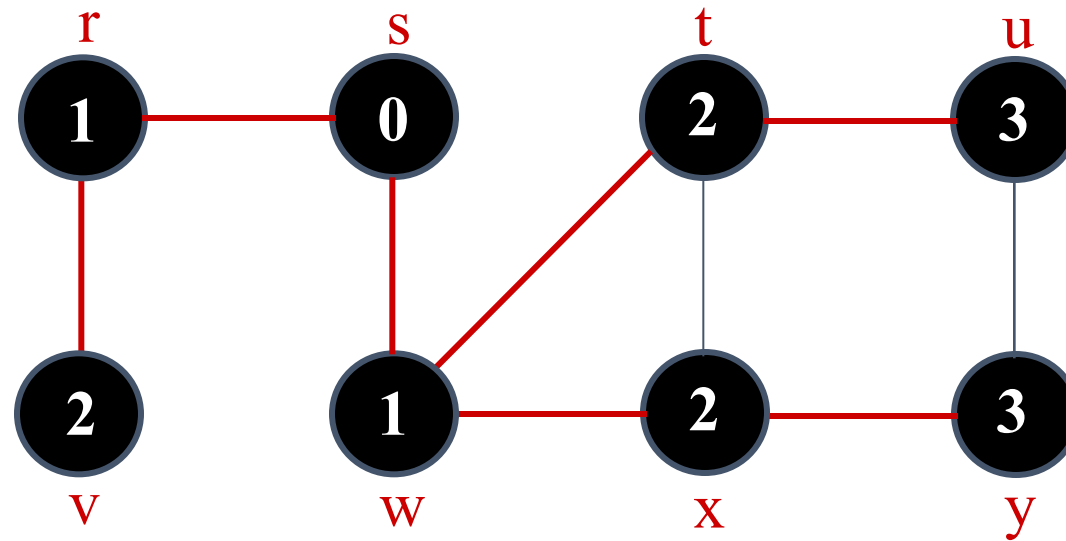
Q: u y
3 3

BFS Example



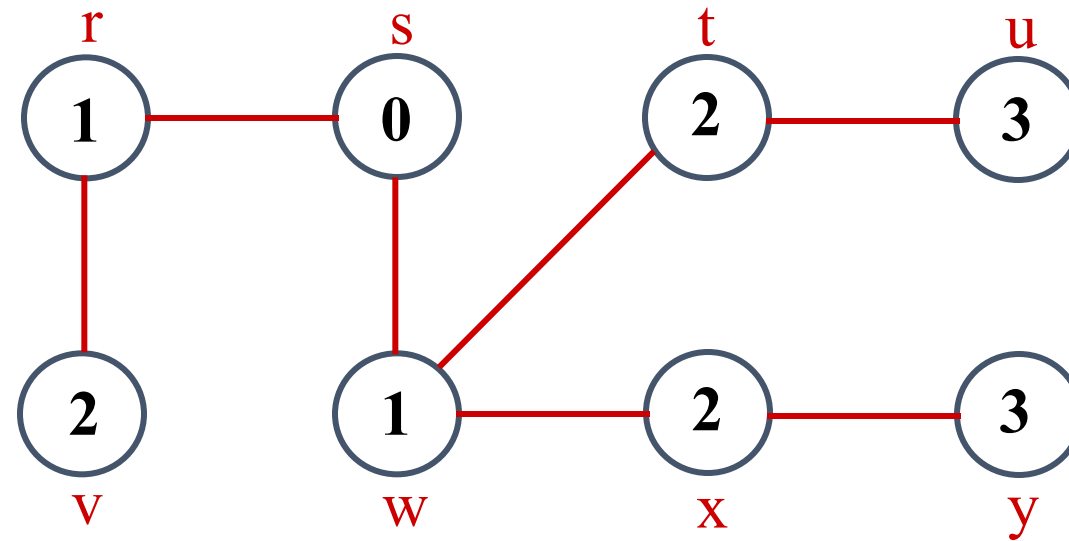
Q: y
3

BFS Example



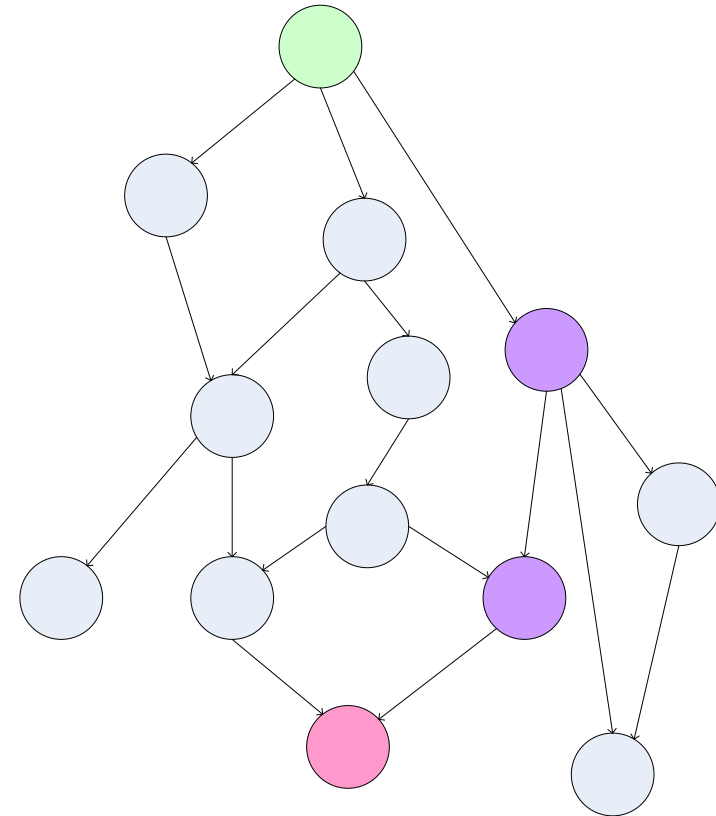
Q: $\emptyset$

Output – BFS Tree



Finding the Shortest Path

- A common graph search application is finding the shortest path from a start node to one or more target nodes
- Commonly done on a single machine with *Dijkstra's Algorithm*
- Can we use BFS to find the shortest path via MapReduce?



Finding the Shortest Path: Intuition

We can define the solution to this problem inductively

- $\text{DistanceTo}(\text{startNode}) = 0$
- For all nodes n directly reachable from startNode , $\text{DistanceTo}(n) = 1$
- For all nodes n reachable from some other set of nodes S ,

$$\text{DistanceTo}(n) = 1 + \min(\text{DistanceTo}(m), m \in S)$$

From Intuition to Algorithm

- A map task receives a node n as a key, and $(D, \text{points-to})$ as its value
 - D is the distance to the node from the start
 - points-to is a list of nodes reachable from n
- For all $p \in \text{points-to}$, emit $(p, D+1)$
- Reduces task gathers possible distances to a given p and selects the minimum one

Blow-up and Termination

Does the algorithm ever terminate?

- Yes! Eventually, routes between nodes will stop being discovered and no better distances will be found. When distance is the same, we stop.
- Mapper should emit (n, D) to ensure that “current distance” is carried into the reducer

Reminder: Dijkstra's algorithm

Dijkstra's algorithm - is a solution to the single-source shortest path problem in graph theory.

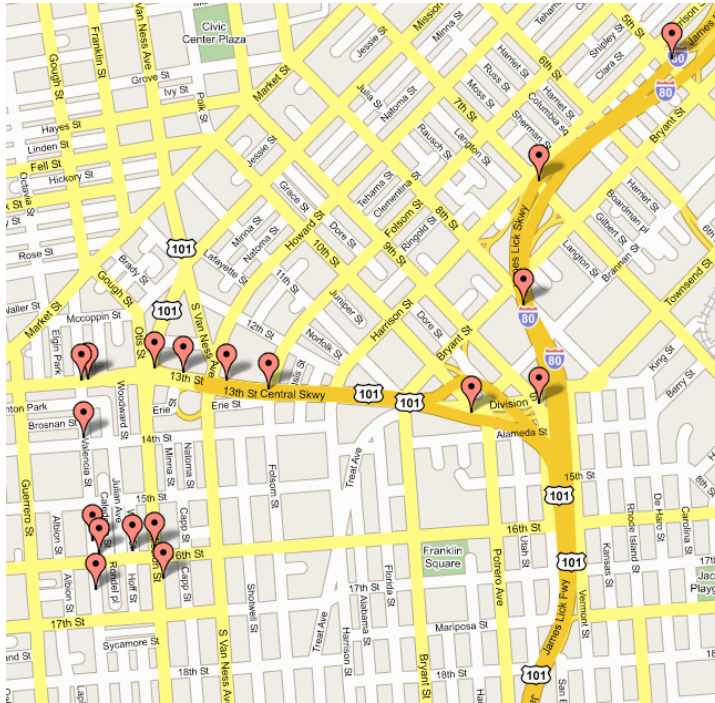
Constraint: all edges must have nonnegative weights.

Input: Weighted graph $G = (V, E)$ and source vertex $v \in V$

Output: Lengths of shortest paths (or the shortest paths themselves) from a given source vertex $v \in V$ to all other vertices

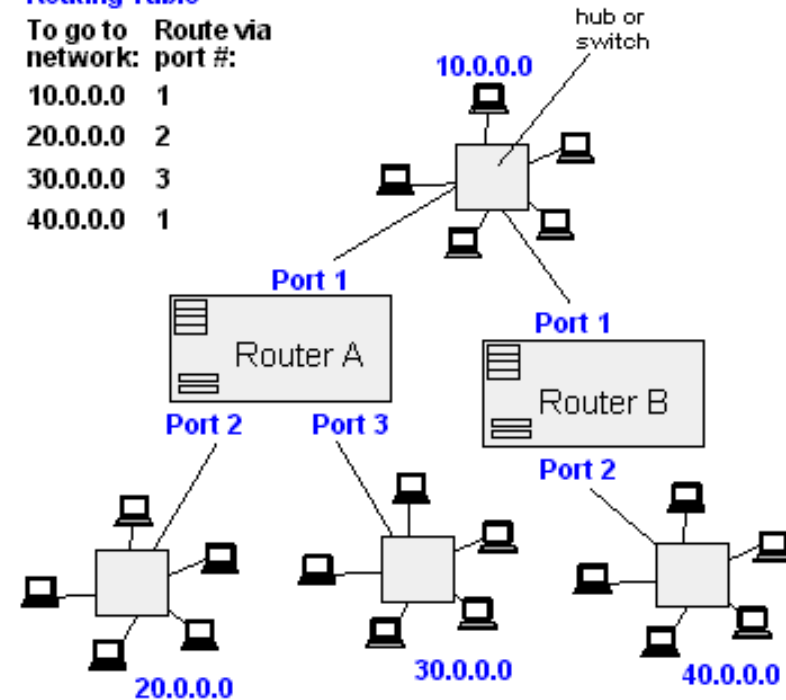
Applications of Dijkstra's Algorithm

From Computer Desktop Encyclopedia
© 1998 The Computer Language Co. Inc.

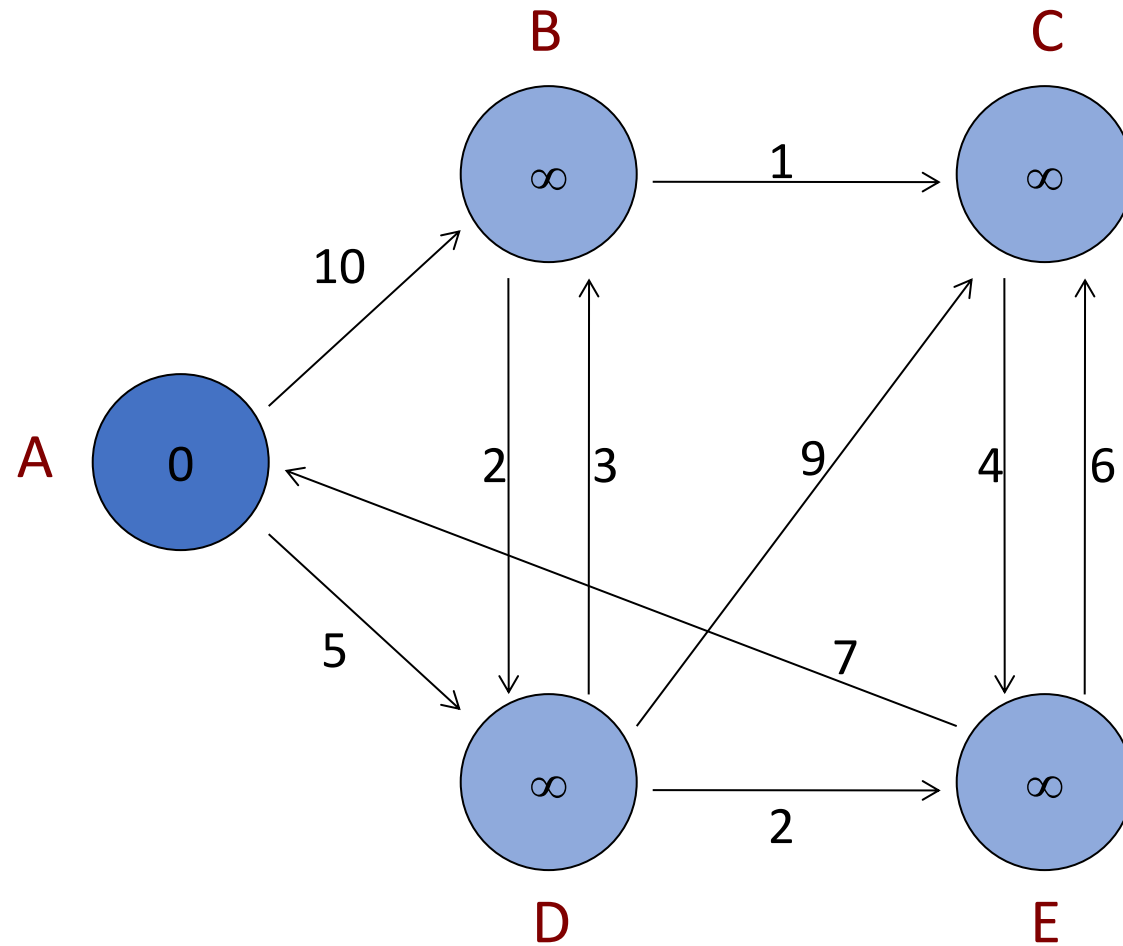


Router A Routing Table

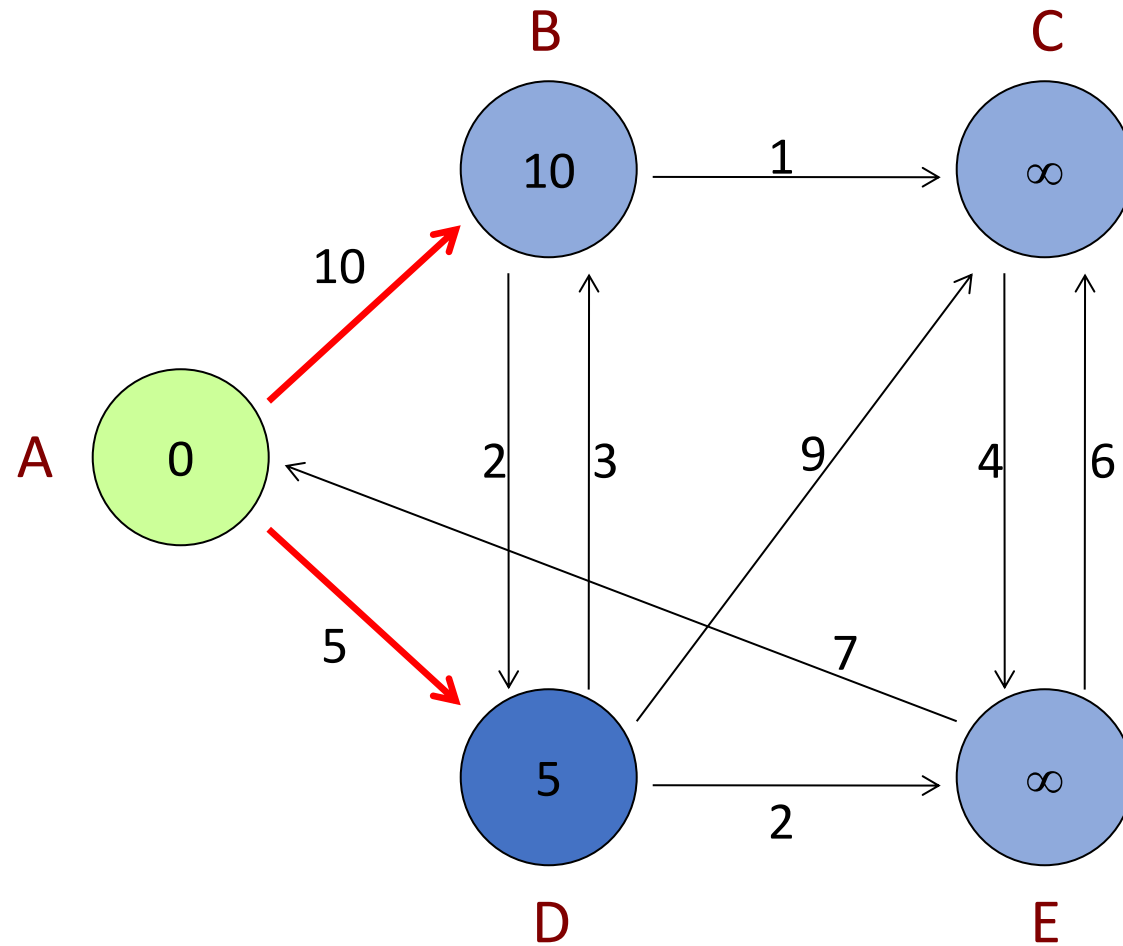
To go to network:	Route via port #:
10.0.0.0	1
20.0.0.0	2
30.0.0.0	3
40.0.0.0	1



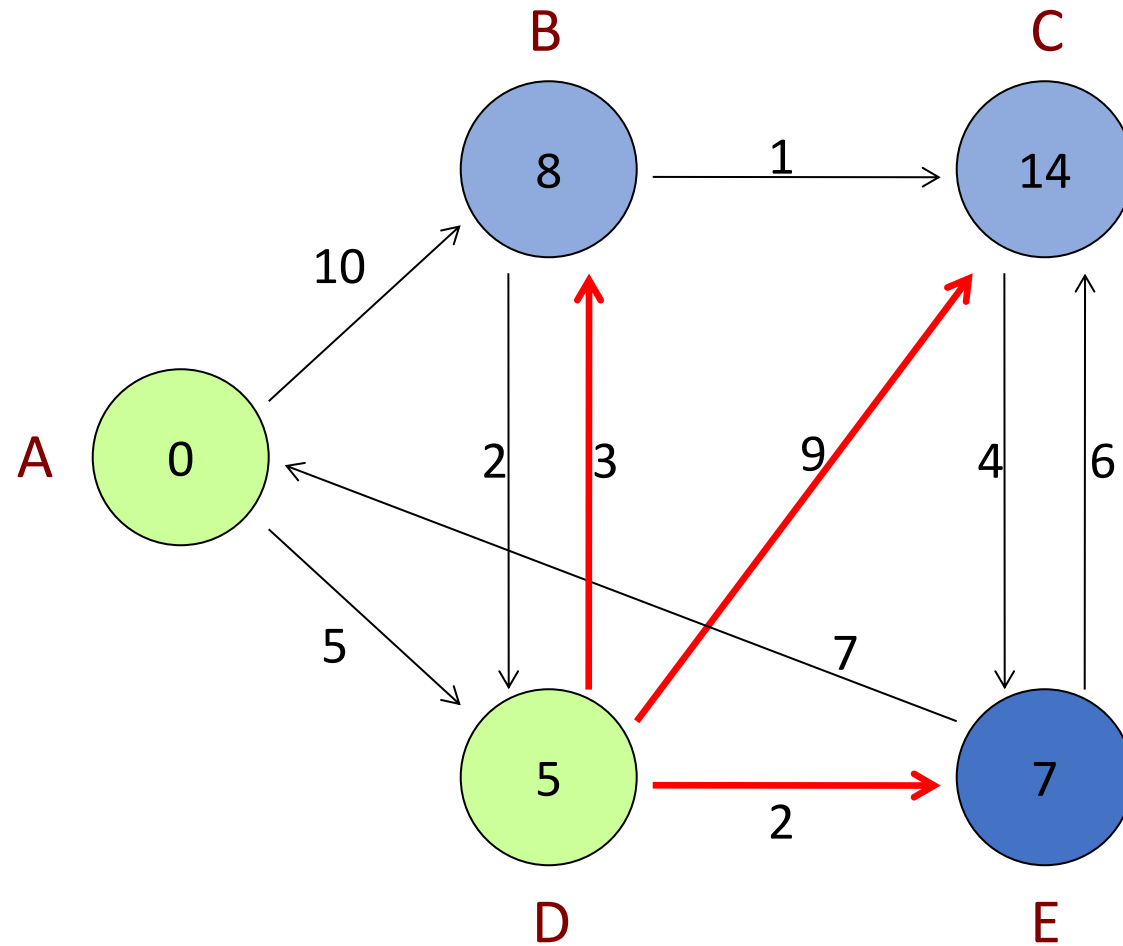
Example: Dijkstra's Algorithm



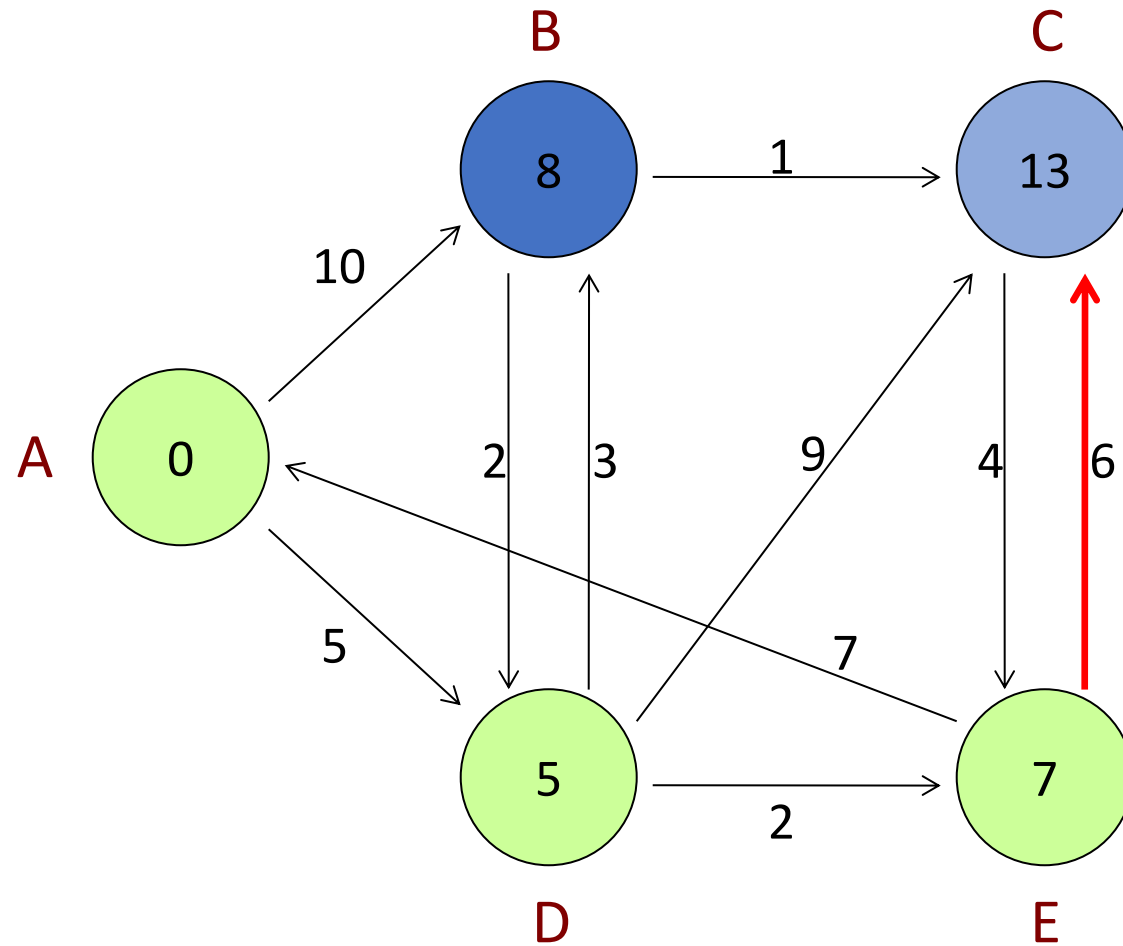
Example: Dijkstra's Algorithm



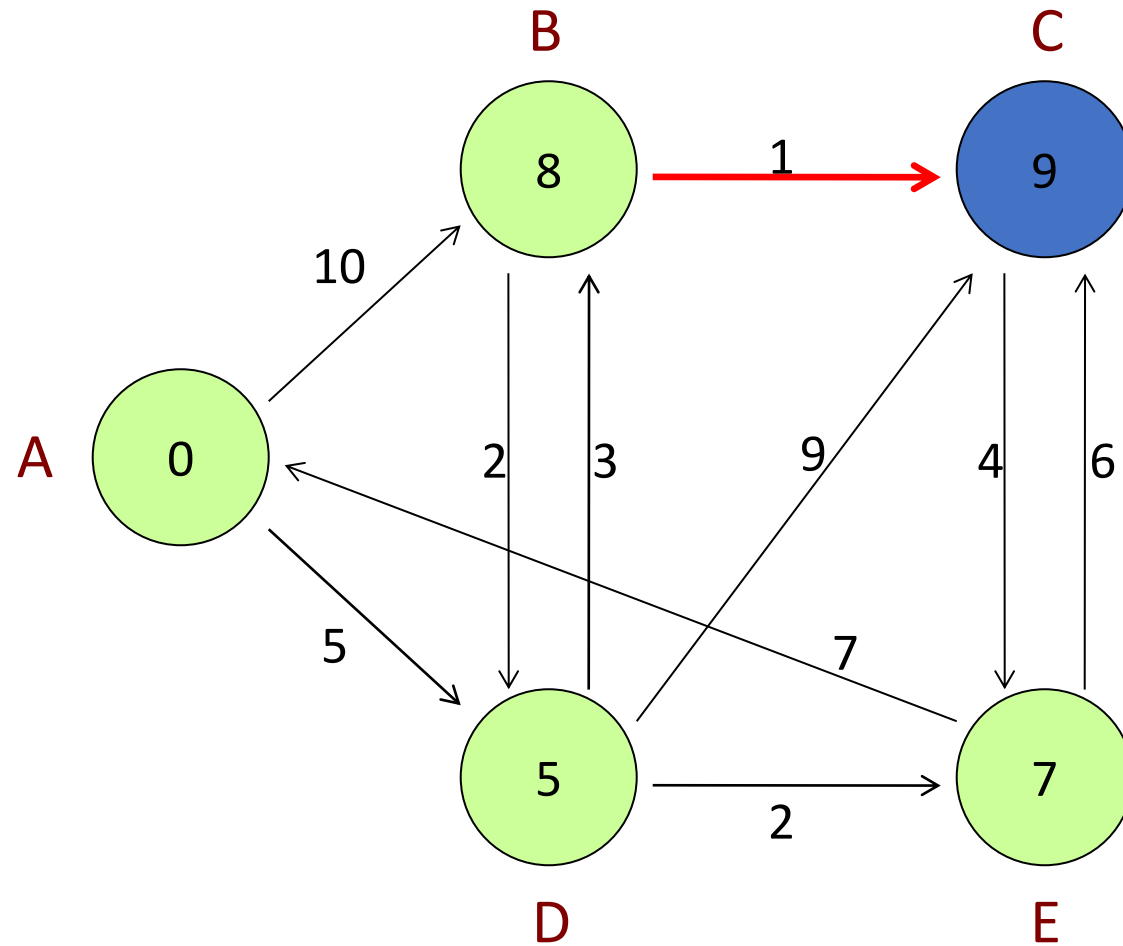
Example: Dijkstra's Algorithm



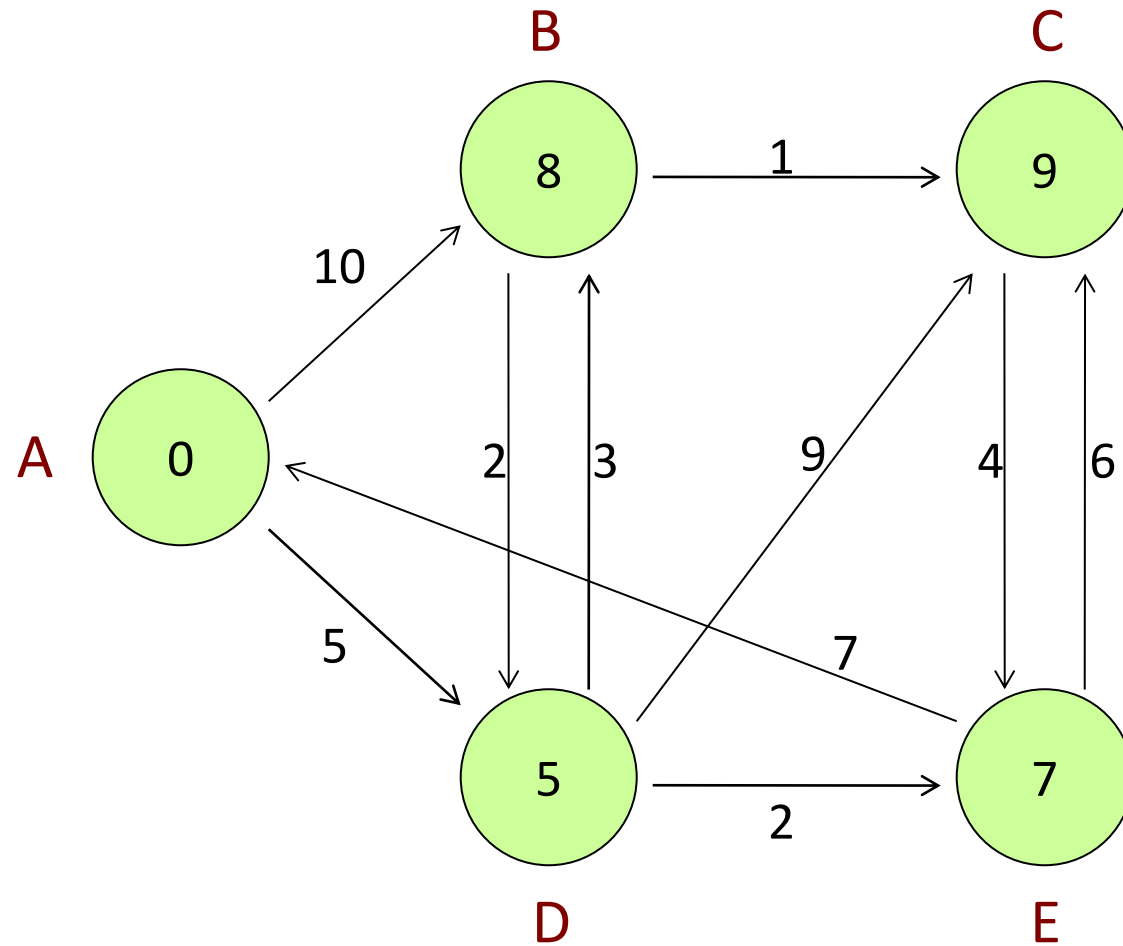
Example: Dijkstra's Algorithm



Example: Dijkstra's Algorithm



Example: Dijkstra's Algorithm



Dijkstra's algorithm

```
dist[s] ← 0                                (distance to source vertex is zero)
for all v ∈ V - {s}
    do dist[v] ← ∞                          (set all other distances to infinity)
S ← ∅                                       (S, the set of visited vertices is initially empty)
Q ← V                                       (Q, the queue initially contains all vertices)
while Q ≠ ∅                                (while the queue is not empty)
do u ← mindistance(Q, dist)                (select the element of Q with the min. distance)
    S ← S ∪ {u}                             (add u to list of visited vertices)
    for all v ∈ neighbors[u]
        do if dist[v] > dist[u] + w(u, v)    (if new shortest path found)
            then d[v] ← d[u] + w(u, v)      (set new value of shortest path)
            (if desired, add traceback code)
return dist
```

Running Times

The simplest implementation is to store vertices in an array or linked list.

Running time: $O(|V|^2 + |E|)$

For sparse graphs, or graphs with very few edges and many nodes, it can be implemented more efficiently storing the graph in an adjacency list using a binary heap or priority queue.

Running time: $O([|V| + |E|] \cdot \log(|V|))$